VIT Agentic AI Workshop Curriculum

Title: Agentic AI in Practice: Build, Review and Control
Speaker: Vedant
Audience: First- to fourth-year IT/CS students, new to agentic AI
When: Saturday 17 October 2026, 11:00-13:00 IST
Duration: 120 minutes, including a five-minute break

Three practical outcomes

Each pair should be able to write a concrete coding-assistant instruction, make one small change in a prepared program and explain its result, and identify an unchecked value in an AI-generated answer. Model/tool/code responsibilities support all three tasks. These are introductory outcomes, not framework mastery.

The classroom deck has 31 slides. The longer 53-slide deck, all five implementations and the full AI Coding Lab remain optional reference material. Use learning_lab/Classroom Follow Along.md for the live exercises. Prepare installations before class; do not turn the first half hour into account troubleshooting.

Two-hour schedule

Time IST	Minutes	Block	Student outcome
11:00-11:10	10	1. Canteen story and model/tool/code	Trace who requests, runs and checks
11:10-11:20	10	2. AI Studio and project checkpoint	See key setup and open a working local demo
11:20-11:40	20	3. Project context and Markdown	Write one useful instruction and read a data record
11:40-12:00	20	4. One agent and one small change	Change a budget in a copy and explain the result
12:00-12:05	5	Break	Resume at 12:05
12:05-12:25	20	5. Review the final answer	Find an unchecked value and propose a behavior check
12:25-12:40	15	6. Read a graph and choose control	Follow one route; understand the framework overview
12:40-13:00	20	7. Exit task and Q&A	Show one instruction, change and review finding

The planned slide slots and schedule.json use the same deadlines. A timed rehearsal is still required. Pair work occupies at least 24 minutes, with ten minutes reserved for closing questions.

One familiar example

The fictional canteen has an available vegetarian sandwich at Rs 60, sold-out poha at Rs 35, an available chicken roll at Rs 75 and a vegetarian dosa at Rs 90. The normal request is vegetarian lunch under Rs 80 plus opening hours. The mock clock reads 12:30 and the canteen closes at 16:00. No orders or payments occur.

Start with a constructed bad answer advertising a nonexistent Paneer Tikka Wrap. Students identify the evidence the assistant needs. Explain model request, application execution and tool result before naming a framework. Use the order-slip/lost-charger analogies and ask students to explain the route aloud.

Guided practice

1. Open the prepared demo folder. Run demo 02 with --offline as the shared local checkpoint. Explicitly label this rule-based stand-in as a simulation. A separately tested Gemini run may be demonstrated.

2. Show a small project map: README, AGENTS.md, docs/PROJECT.md, data/menu.json, code and tests. Teach headings, lists and one command block. Each pair writes five concrete instruction lines, then adds its best line to the supplied AGENTS.md. Discuss one instruction file in practice; mention other client conventions briefly.

3. Read one JSON menu record. Separate changing records from coding instructions and program behavior.

4. Copy 02_react_agent.py to practice_agent.py in the same demo folder. Change only the QUESTION budget from Rs 80 to Rs 30. Run the copy, inspect the one-line diff and explain why no food qualifies. Use manual editing or a coding assistant. Keep data/tools unchanged. Advanced pairs try sold_out or menu_timeout.

5. In pairs, review the deliberately unsafe return of reply["message"] after only the dish list was checked. Name the unchecked value and propose an input/output check. Demonstrate the corrected result with demo 04, without teaching its entire implementation.

Students show three pieces of evidence at the end: one useful instruction, a changed line plus observed answer, and a review finding/check. If many cannot explain who executes a tool, revisit that diagram and shorten the framework overview.

AI Studio setup, bounded to ten minutes

Use the Google AI Studio interface to show sign-in, API Keys, the default project/key or key creation, and checking free-tier/model access. Google Cloud Console provisioning, billing upgrades and credit purchases are outside this session. Describe the free allocation as free-tier quota; do not promise a cash credit amount, unlimited calls or eligibility for every account.

Show .env.example with a blank key. Students opting into live use privately copy their own key to .env as GOOGLE_API_KEY and run check_setup.py. Signing into Antigravity and configuring the Python demo are separate steps. A real API call consumes quota. AI Studio is 18+; account-limited students use offline/paper participation. If setup blocks a pair, continue locally. Do not share or project a real key.

The live Gemini demos must be checked separately by the presenter. Offline tests do not establish account quota, model availability or live tool-calling behavior.

Keep the framework comparison short

Use fifteen minutes for a visual LangGraph explanation, one normal/failure route and a brief architecture discussion. State carries information; nodes do work; edges route execution. LangGraph can contain flexible agents. ReAct is a pattern. CrewAI offers crews and flows. Framework choice does not determine autonomy by itself.

Move full graph construction, multi-agent implementation, CrewAI setup, the refund extension, Ainaa recording and deployment to optional reference material. The earlier speaker already covers Azure, a LangChain agent and MCP creation. Do not repeat that provisioning.

Practical preparation and fallback

Before the workshop, students install Python 3.11 and one editor, extract the supplied kit and follow demo/README.md to install dependencies. Antigravity is optional for the guided coding-assistant step; an existing editor supports all core manual tasks. Use one laptop per pair when needed. The follow-along includes Windows and macOS/Linux commands.

Ask for reliable Wi-Fi, power and setup helpers. Anyone without a working device can use a paired laptop or the printed trace/review task. Keep saved outputs ready. Protect the student tasks and Q&A if late. Cut extra live runs and framework vocabulary first. Preserve the honest distinction between real Gemini, injected mistakes and the offline stand-in.

Official references

- Gemini API keys
- Gemini free tier and billing
- AI Studio access requirements
- Antigravity project rules
- LangGraph overview
- CrewAI crews and flows